Experiment 9: Ansible Lab Report

1. Objective

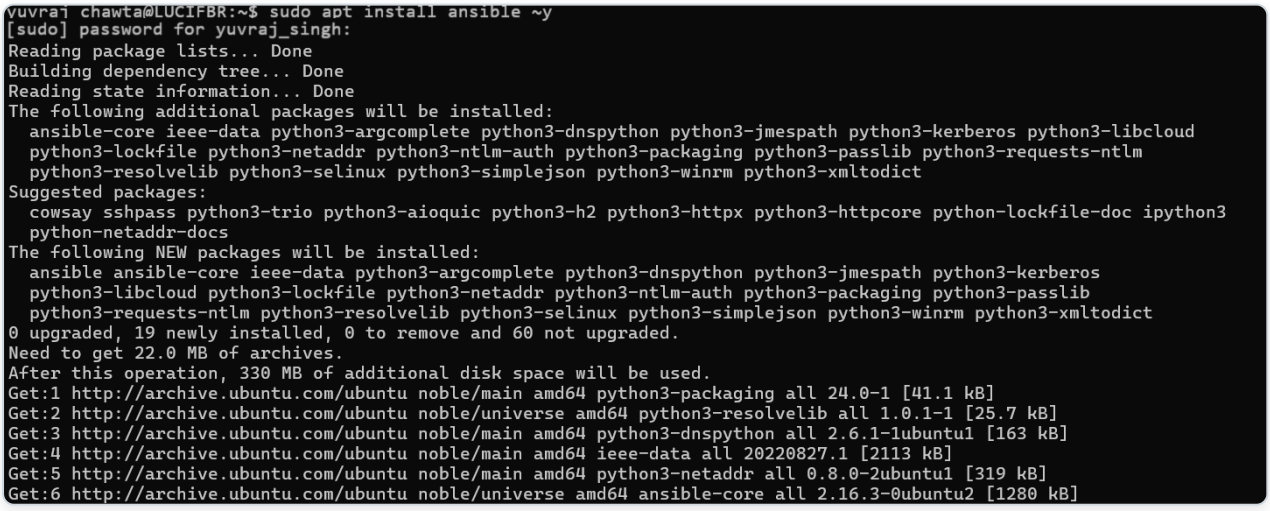
To understand and implement infrastructure configuration management using Ansible. The experiment demonstrates how to provision, configure, and manage multiple remote servers (simulated using Docker containers) simultaneously from a single control node without installing any agent software on the managed nodes.

2. Theory

Ansible is an open-source automation tool used for configuration management, application deployment, and orchestration.
* **Agentless Architecture:** It does not require any background agents on the target machines, relying entirely on standard SSH for Linux environments.
* **Idempotency:** Playbooks can be run multiple times safely; Ansible only makes changes if the system does not match the desired state.
* **Playbooks:** Uses human-readable YAML syntax to define tasks, ensuring Infrastructure as Code (IaC) principles.
* **Inventory:** A file defining the list of managed nodes (target servers) that Ansible will connect to.

3. Environment Setup

Prerequisites: * Windows Subsystem for Linux (WSL) running Ubuntu (Control Node).
* Docker Desktop configured to integrate with WSL.



```
yuvraj_singh@LUCIF3R $ ansible ~-version
ansible [core 2.16.3]
  config file = None
  configured module search path ['/home/yuvraj_singh/.ansible/plugins_ modules lusr/share/ansible/plugins/modules
]
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location Ihome/yuvraj_singh/ ansible/collections lusr/sharelansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

```
yuvraj_singh@LUCIF3R: $ ansible localhost ~m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

3.1 Generating SSH Keys

To allow Ansible to connect to the target nodes without passwords, an SSH key pair was generated on the control node.

```
Generating public/private yuvraj_singh@LUCIF3R: $ ssh-keygen rsa key pair ~t rsa -b 4096
Enter file in which to save the key (/home/yuvraj_singh/ sshlid_rsa)  :
Created directory /home/yuvraj_singh/ ssh    '.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in Ihome/yuvraj_singh/ sshlid_rsa
Your public key has been saved in /home/yuvraj_singh/ sshlid_rsa pub
The key fingerprint is:
SHA256:XqXYAC29H1532Dkroec/klg796hOLDR7T/ze4YtMSZE yuvraj_singh@LUCIF3R
The key's randomart image is:
+---[RSA 4096]----+
|     .o          |
|     ..o         |
|      ...   .  o . |
|       .=.o. + = .|
|       .oSooo o E |
|        .o..++ + .|
|           o=+= o |
|           .+Oo=.o|
|            .oX*B*|
+----[SHA256]-----+
```

```
yuvraj_ singh@LUCIF3R: $ cp sshlid rsa pub  .
yuvraj_singh@LUCIF3R cp ~/ .sshlid_ rsa     .
yuvraj singh@LUCIF3R: Ls
Dockerfile yuvraj_singh@LUCIF3R: env-Lab exp6 $ flask-env-app id_rsa id_rsa.pub my-app myapp-data nginx-config
```

3.2 Building the Target Docker Image

A custom `ubuntu-server` Docker image was created to simulate the managed nodes. It installs `openssh-server`, Python (required by Ansible), and imports the generated SSH keys.

Dockerfile:

```
yuvraj_singh@LUCIF3R:  ~
  GNU nano 7.2                                                    Dockerfile
FROM ubuntu


RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config


# Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys (note: this is not secure for production!)
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

# Set proper permissions for keys
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# Fix for SSH login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

# Expose SSH port
EXPOSE 22

# Start SSH service when container starts
CMD ["/usr/sbin/sshd", "-D"]
```

Build Command:

```
yuvraj_singh@LUCIF3R: $ docker build -t ubuntu-server    .
[+] Building 123.7s (16/16) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                             0.0s
 => => transferring dockerfile: 1.10kB                                                                           0.0s
 => [internal] load metadata for docker.io/library/ubuntu:latest                                                 3.1s
 => [auth] library/ubuntu:pull token for registry-1.docker.io                                                    0.0s
 => [internal] load .dockerignore                                                                                0.0s
 => => transferring context: 2B                                                                                  0.0s
 => [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194eb  6.5s
 => => resolve docker.io/library/ubuntu:latest@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc  0.0s
 => => sha256:b40150c1c2717d324cdb17278c8efdfa4dfcd2ffe083e976f0bcedf31115f081 29.73MB / 29.73MB                 6.0s
 => => extracting sha256:b40150c1c2717d324cdb17278c8efdfa4dfcd2ffe083e976f0bcedf31115f081                        0.5s
 => [internal] load build context                                                                                0.0s
 => => transferring context: 4.21kB                                                                              0.0s
 => [ 2/10] RUN apt update -y                                                                                   11.3s
 => [ 3/10] RUN apt install -y python3 python3-pip openssh-server                                               84.4s
 => [ 4/10] RUN mkdir -p /var/run/sshd                                                                           0.3s
 => [ 5/10] RUN mkdir -p /run/sshd &&     echo 'root:password' | chpasswd &&     sed -i 's/#PermitRootLogin prohi  0.3s
 => [ 6/10] RUN mkdir -p /root/.ssh &&     chmod 700 /root/.ssh                                                  0.3s
 => [ 7/10] COPY id_rsa /root/.ssh/id_rsa                                                                        0.0s
 => [ 8/10] COPY id_rsa.pub /root/.ssh/authorized_keys                                                           0.0s
 => [ 9/10] RUN chmod 600 /root/.ssh/id_rsa &&     chmod 644 /root/.ssh/authorized_keys                          0.3s
 => [10/10] RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/ssh  0.3s
 => exporting to image                                                                                          16.7s
 => => exporting layers                                                                                         13.2s
 => => exporting manifest sha256:981d29dbe643d90e4d20a6c60ddd778443f08d9499cbe6cf41270309c2e39196                 0.0s
 => => exporting config sha256:bc05668b1712174b190dc5c5a13b3e4967540cd1416bdb215c33bed214cbb222                  0.0s
 => => exporting attestation manifest sha256:6372187a2dbdf1ebbf34ca774d0dd90c0093405501603ad11f291a495a481c40    0.0s
 => => exporting manifest list sha256:c487c16ad3a9bd0cc966b86050d175cbf42efac0d04f88c585d5976aca809595           0.0s
 => => naming to docker.io/library/ubuntu-server:latest                                                          0.0s
 => => unpacking to docker.io/library/ubuntu-server:latest                                                       3.5s
```

building a test container.

```
yuvraj_singh@LUCIF3R:~$ docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
988fff003f84699944bfdb86416eca4dab06ca704ec76644a10692921f6c151c
```

```
yuvraj_singh@LUCIF3R: $ docker inspect -f {{range. NetworkSettings.Networks}} { { . IPAddress}} {{end}} ' ssh-test-server
172.17.0.2
```

```
yuvraj_chala@LUCIF3R: $ ssh -i ~/.sshlid_rsa root@localhost 2222
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Apr 29 13:09:38 2026 from 172.17.0.1
root@988fff003f84:~#
```

removing the container.

```
yuvraj_singh@LUCIF3R: docker stop ssh-test-server
 ssh-test-server
yuvraj_singh@LUCIF3R:~$ docker rm ssh-test-server
 Error response from daemon: No such container: ssh-test-server
yuvraj_singh@LUCIF3R: $
```

4. Implementation Steps

4.1 Provisioning the Managed Nodes

Four Docker containers (`server1` to `server4`) were spun up in the background to act as the infrastructure.

```
yuvraj_singh@LUCIF3R:~$ for in {1..4}; do
  echo -e "\n Creating server${i}\n"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo -e "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})
"
done

 Creating server1

7b08a10c46d9958c3d28dd4e75f07c5b9d451c500e903af5806ef29ed46723d1
IP of server1 is 172.17.0.2

 Creating server2

dc7ddafe820c0fc5401474e601f94ec6ade9fb8a5d8b8961836105ead632a1f9
IP of server2 is 172.17.0.3

 Creating server3

18e0ef0a048e2ab31c8f235896ca0c26aa8ca28ef39cecf9ed488de3364eb813
IP of server3 is 172.17.0.4

 Creating server4

d224e70ff68df146bc4dc0a49ebd2969fc477206ca1509dd711adec8db3697d2
IP of server4 is 172.17.0.5
```

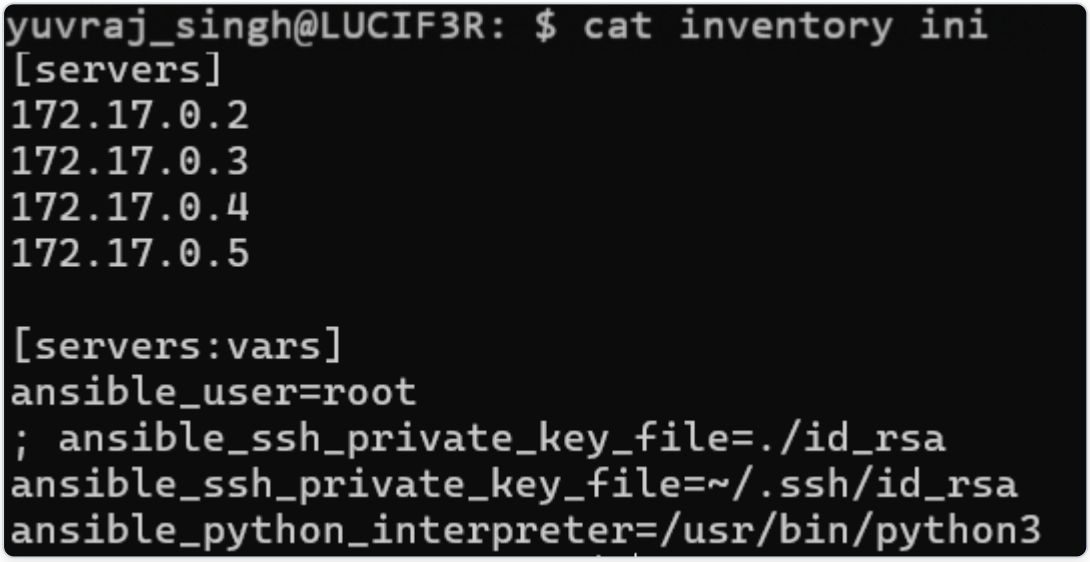
4.2 Creating the Ansible Inventory

An `inventory.ini` file was dynamically generated to tell Ansible how to reach the containers using their internal Docker IP addresses and the custom SSH key.

```
yuvraj singh@LUCIF3R:~$ Get container IPs
echo "[servers]" > inventory.ini
for i in {1..4}; do
  docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

# Add inventory variables
cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
yuvraj_singh@LUCIF3R:~$ ls
Dockerfile  env-lab  exp6  flask-env-app  id_rsa  id_rsa.pub  inventory.ini  my-app  myapp-data  nginx-config
```

4.3 Testing Connectivity (Ping Module)

To verify Ansible could communicate with all four nodes, the `ping` module was used. Strict host key checking was temporarily disabled to bypass the simultaneous SSH fingerprint prompts for the new containers.



4.4 Creating and Executing the Playbook

A playbook was created to automate the configuration of all four servers simultaneously: updating apt packages, installing tools (`vim` , `htop` , `wget`), and writing a custom text file.

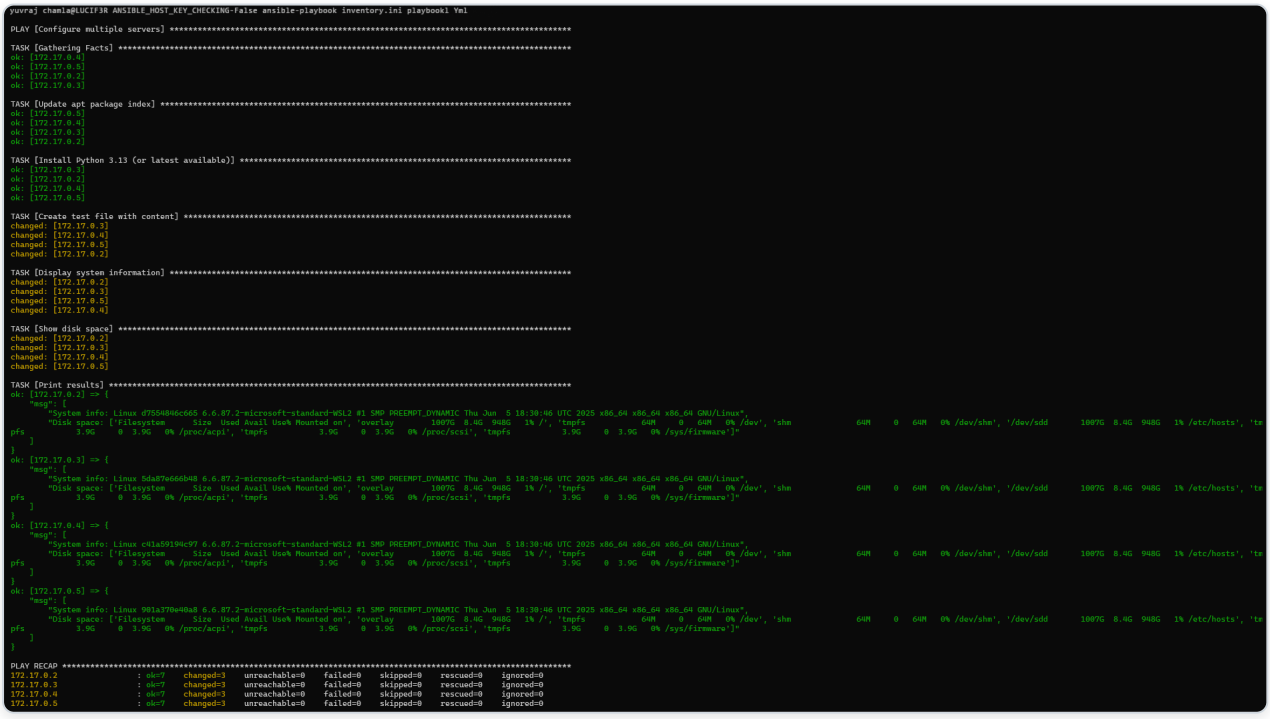
playbook1.yml:

yuvraj_singh@LUCIF3R: ~

GNU nano 7.2 playbook1.yml

```yaml
--- # it should start with three dash only
- name: Configure multiple servers
  hosts: servers
  become: yes

  tasks:
    - name: Update apt package index
      apt:
        update_cache: yes

    - name: Install Python 3.13 (or latest available)
      apt:
        name: python3
        state: latest

    - name: Create test file with content
      copy:
        dest: /root/test_file.txt
        content: |
          This is a test file created by Ansible
          Server name: {{ inventory_hostname }}
          Current date: {{ ansible_date_time.date }}

    - name: Display system information
      command: uname -a
      register: uname_output

    - name: Show disk space
      command: df -h
      register: disk_space

    - name: Print results
      debug:
        msg:
          - "System info: {{ uname_output.stdout }}"
          - "Disk space: {{ disk_space.stdout_lines }}"
```

Execution Command:



4.5 Verification and Cleanup

Changes were verified across the cluster using Ansible ad-hoc commands to read the newly created text file on all nodes simultaneously.

```
yuvraj chaw a@LUCIFBR: ansible all ~1 inventory ini ~mcommand ~a echo Hello trom Ansible /root ansible_test tx
172.17.0.4 | CHANGED | rc=0 >>
Hello from Ansible > /root/ansible_test.txt
172.17.0.5 | CHANGED | rc=0 >>
Hello from Ansible > /root/ansible_test.txt
172.17.0.3 | CHANGED | rc=0 >>
Hello from Ansible > /root/ansible_test.txt
172.17.0.2 | CHANGED | rc=0 >>
Hello from Ansible > /root/ansible_test.txt
```

Once verified, the infrastructure was torn down by stopping and removing the Docker containers:

```
yuvraj_singh@LUCIF3R:~$ for in {1..4}; do
  docker stop server${i}
done
server1
server2
server3
server4
```

5. Conclusion

This experiment successfully demonstrated the core workflow of Ansible. By using a single control node, we were able to seamlessly provision, connect to, and configure multiple remote systems simultaneously using an automated YAML playbook. This highlights Ansible's value in reducing manual configuration drift and improving scalability in